Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

Cursul 4

20 Octombrie 2025

4 Recurrent neural networks

- so far, we encountered two types of data: tabular data and image data
- for the latter, we designed specialized layers to take advantage of the regularity in them
- in other words, if we were to permute the pixels in an image, it would be much more difficult to reason about its content
- most importantly, so far, we assumed that our data are all drawn from some distribution, and all the examples are independently and identically distributed (i.i.d.)

4 Recurrent neural networks

- unfortunately, this is not true for most data
- for instance, the words in this sentence are written in sequence, and it would be quite difficult to decipher its meaning if they were permuted randomly
- likewise, image frames in a video, the audio signal in a conversation, and the browsing behavior on a website, all follow a *sequential* order
- it is thus reasonable to assume that specialized models for such data will do better at describing them

4 Recurrent neural networks

- another issue arises from the fact that we might not only receive a sequence as an input, but rather might be expected to continue the sequence
- for instance, the task could be to continue the series $2, 4, 6, 8, 10, \ldots$
- this is quite common in time series analysis, to predict the stock market, the fever curve of a patient, or the acceleration needed for a race car
- again, we want to have models that can handle such data
- in short, while CNNs can efficiently process spatial information, *recurrent neural networks* (*RNNs*) are designed to better handle *sequential* information
- RNNs introduce state variables to store past information, together with the current inputs, to determine the current outputs

4.1 Sequence models

- we need statistical tools and new deep neural network architectures to deal with sequence data
- to keep things simple, we use the stock price (FTSE 100 index) illustrated in Figure 1, as an example

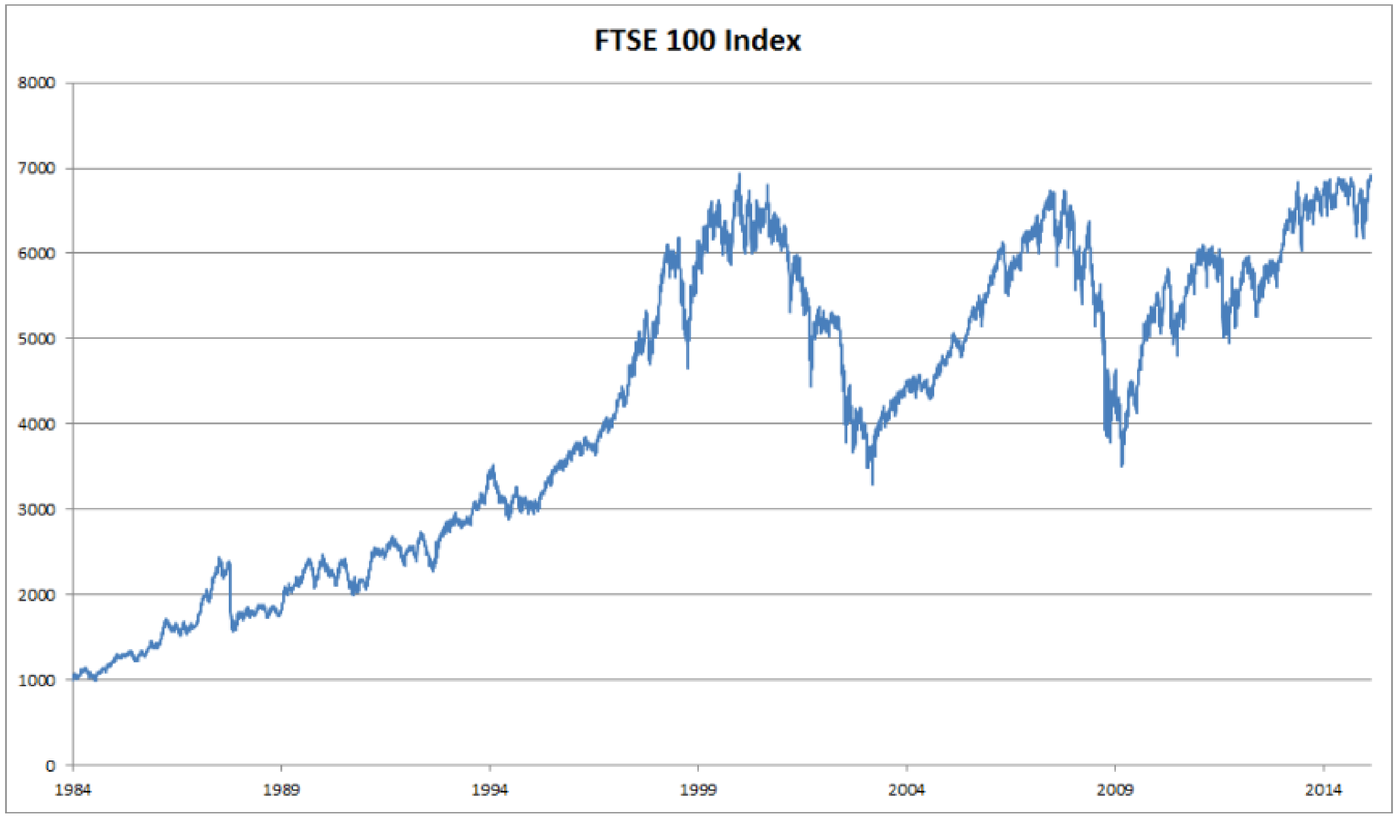


Figure 1: FTSE 100 index over about 30 years.

4.1 Sequence models

- let us denote the prices by x_t, i.e., at *time step* $t \in \mathbb{Z}^+$ we observe price x_t
- note that, for sequences in this chapter, t will typically be discrete and vary over integers or its subset
- suppose that a trader who wants to do well in the stock market on day t predicts x_t via:

$$x_t \sim P(x_t | x_{t-1}, \ldots, x_1).$$

- in order to achieve this, our trader could use a regression model such as the one that we discussed in Chapter 1

4.1 Sequence models

- there is just one major problem: the number of inputs $x_{t-1}, \ldots, x_1$ varies, depending on t
- that is to say, the number increases with the amount of data that we encounter, and we will need an approximation to make this computationally tractable
- much of what follows in this chapter will revolve around how to estimate $P(x_t|x_{t-1}, \ldots, x_1)$ efficiently
- in a nutshell, it boils down to two strategies, as follows

4.1 Sequence models

- first, assume that the potentially rather long sequence $x_{t-1}, \ldots, x_1$ is not really necessary
- in this case, it might be sufficient to use some timespan of length τ, and only use the $x_{t-1}, \ldots, x_{t-\tau}$ observations
- the immediate benefit is that, now, the number of arguments is always the same, at least for $t > \tau$
- this allows us to train a deep network, as indicated above
- such models will be called *autoregressive models*, as they literally perform regression on themselves

4.1 Sequence models

- the second strategy, shown in Figure 2, is to keep some summary h_t of the past observations, and, at the same time, update h_t in addition to the prediction $\hat{x}_t$
- this leads to models that estimate x_t with $\hat{x}_t = P(x_t|h_t)$ and, moreover, updates of the form $h_t = g(h_{t-1}, x_{t-1})$
- since h_t is not observed in the data, these models are also called *latent autoregressive models*

4.1 Sequence models

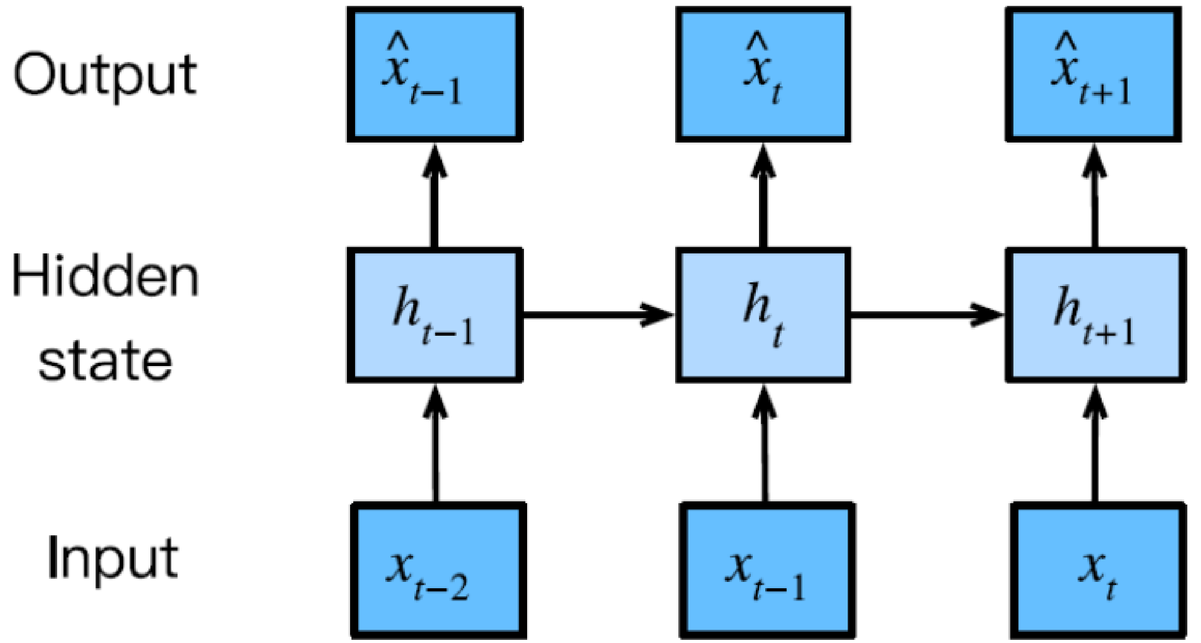


Figure 2: A latent autoregressive model.

4.1 Sequence models

- both cases raise the obvious question of how to generate training data
- we typically use historical observations to predict the next observation, given the ones up to right now
- obviously, we do not expect time to stand still; however, a common assumption is that, while the specific values of x_t might change, at least the dynamics of the sequence itself will not
- this is reasonable, since novel dynamics are just that, novel, and thus not predictable using data that we have so far
- statisticians call dynamics that do not change *stationary*
- regardless of what we do, we will thus get an estimate of the entire sequence via:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t | x_{t-1}, \ldots, x_1).$$

4.1 Sequence models

- note that the above considerations still hold if we deal with discrete objects, such as words, rather than continuous numbers
- the only difference is that, in such a situation, we need to use a classifier rather than a regression model to estimate $P(x_t|x_{t-1}, \ldots, x_1)$
- recall the approximation that, in an autoregressive model, we use only $x_{t-1}, \ldots, x_{t-\tau}$ instead of $x_{t-1}, \ldots, x_1$ to estimate x_t
- whenever this approximation is accurate, we say that the sequence satisfies the *Markov condition*
- in particular, if $\tau = 1$, we have a *first-order Markov model*, and $P(x)$ is given by:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t|x_{t-1}) \text{ where } P(x_1|x_0) = P(x_1).$$

4.1 Sequence models

- such models are particularly useful whenever x_t assumes only a discrete value, since, in this case, dynamic programming can be used to compute values along the chain exactly
- for instance, we can compute $P(x_{t+1}|x_{t-1})$ efficiently:

$$\begin{aligned} P(x_{t+1}|x_{t-1}) &= \frac{\sum_{x_t} P(x_{t+1}, x_t, x_{t-1})}{P(x_{t-1})} \\ &= \frac{\sum_{x_t} P(x_{t+1}|x_t, x_{t-1}) P(x_t, x_{t-1})}{P(x_{t-1})} \\ &= \sum_{x_t} P(x_{t+1}|x_t) P(x_t|x_{t-1}), \end{aligned}$$

  by using the fact that we only need to take into account a very short history of past observations: $P(x_{t+1}|x_t, x_{t-1}) = P(x_{t+1}|x_t)$
- going into details of dynamic programming is beyond the scope of this section
- control and reinforcement learning algorithms use such tools extensively

4.1 Sequence models

- in principle, there is nothing wrong with unfolding $P(x_1, \ldots, x_T)$ in reverse order
- after all, by conditioning, we can always write it via:

$$P(x_1, \ldots, x_T) = \prod_{t=T}^{1} P(x_t | x_{t+1}, \ldots, x_T).$$

- in fact, if we have a Markov model, we can obtain a reverse conditional probability distribution, too
- in many cases, however, there exists a natural direction for the data, namely going forward in time
- it is clear that future events cannot influence the past

4.1 Sequence models

- hence, if we change x_t, we may be able to influence what happens for x_{t+1} going forward, but not the converse
- that is, if we change x_t, the distribution over past events will not change
- consequently, it should to be easier to explain $P(x_{t+1}|x_t)$ rather than $P(x_t|x_{t+1})$
- for instance, it has been shown that, in some cases, we can find $x_{t+1} = f(x_t) + \epsilon$ for some additive noise ϵ, whereas the converse is not true
- this is great news, since it is typically the forward direction that we are interested in estimating

4.1 Sequence models

- we have reviewed statistical tools and prediction challenges for sequence data; such data can take many forms
- specifically, text is one of the most popular examples of sequence data
- for example, an article can be simply viewed as a sequence of words, or even a sequence of characters
- to facilitate our future experiments with sequence data, we will explain common preprocessing steps for text
- usually, these steps are:
  1. Load text as strings into memory.
  2. Split strings into *tokens* (e.g., words and characters).
  3. Build a *table* of vocabulary to map the split tokens to numerical indices.
  4. Convert text into sequences of numerical indices so they can be manipulated by models easily.

4.1 Sequence models

- each text sequence is split into a list of tokens
- a *token* is the basic unit in text; in the end, a list of token lists are obtained, where each token is a string
- the string type of the token is inconvenient to be used by models, which take numerical inputs
- now, we build a dictionary, often called a *vocabulary* as well, to map string tokens into numerical indices starting from 0

4.1 Sequence models

- to do so, we first count the unique tokens in all the documents from the training set, namely a *corpus*, and then assign a numerical index to each unique token according to its frequency
- rarely appeared tokens are often removed to reduce the complexity
- any token that does not exist in the corpus, or has been removed, is mapped into a special unknown token “<unk>”
- we can also add a list of reserved tokens, such as “<pad>” for padding, “<bos>” to represent the beginning of a sequence, and “<eos>” for the end of a sequence

4.1 Sequence models

- so, we saw how to map text data into tokens, where these tokens can be viewed as a sequence of discrete observations, such as words or characters
- assume that the tokens in a text sequence of length T are $x_1, x_2, \ldots, x_T$
- then, in the text sequence, x_t $(1 \leq t \leq T)$ can be considered as the observation or label at time step t
- given such a text sequence, the goal of a *language model* is to estimate the joint probability of the sequence $P(x_1, x_2, \ldots, x_T)$

4.1 Sequence models

- language models are incredibly useful
- for instance, an ideal language model would be able to generate natural text just on its own, simply by drawing one token at a time $x_t \sim P(x_t|x_{t-1}, \ldots, x_1)$
- all text emerging from such a model would pass as natural language, e.g., English text
- furthermore, it would be sufficient for generating a meaningful dialog, simply by conditioning the text on previous dialog fragments
- clearly, we are still very far from designing such a system, since it would need to *understand* the text, rather than just generate grammatically correct content

4.1 Sequence models

- let us now apply Markov models to language modeling
- a distribution over sequences satisfies the Markov property of first order if $P(x_{t+1}|x_t, \ldots, x_1) = P(x_{t+1}|x_t)$
- higher orders correspond to longer dependencies
- this leads to a number of approximations that we could apply to model a sequence:

$$\begin{aligned} P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2)P(x_3)P(x_4), \\ P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2|x_1)P(x_3|x_2)P(x_4|x_3), \\ P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)P(x_4|x_2, x_3). \end{aligned}$$

- the probability formulas that involve one, two, three, and, in general, n variables are typically referred to as *unigram*, *bigram*, *trigram*, and *n-gram* models, respectively

4.2 Recurrent neural networks

- in n-gram models, the conditional probability of word x_t at time step t only depends on the $n-1$ previous words
- if we want to incorporate the possible effect of words earlier than time step $t-(n-1)$ on x_t, we need to increase n
- however, the number of model parameters would also increase exponentially with it, as we need to store $|\mathcal{V}|^n$ numbers for a vocabulary set $\mathcal{V}$
- hence, rather than modeling $P(x_t|x_{t-1},\ldots,x_{t-n+1})$, it is preferable to use a latent variable model:

$$P(x_t|x_{t-1},\ldots,x_1) \approx P(x_t|h_{t-1}),$$

  where h_{t-1} is a *hidden state* (also known as a *hidden variable*) that stores the sequence information up to time step $t-1$
- in general, the hidden state at any time step t could be computed based on both the current input x_t, and the previous hidden state h_{t-1}:

$$h_t = f(x_t, h_{t-1}). \tag{1}$$

4.2 Recurrent neural networks

- for a sufficiently powerful function f in (1), the latent variable model is not an approximation
- after all, h_t may simply store all the data it has observed so far
- however, it could potentially make both computation and storage expensive
- recall that we have discussed hidden layers with hidden units in Chapter 2
- it is noteworthy that hidden layers and hidden states refer to two very different concepts
- hidden layers are, as explained, layers that are hidden from view, on the path from input to output
- hidden states are, technically speaking, *inputs* to whatever we do at a given step, and they can only be computed by looking at data at previous time steps

4.2 Recurrent neural networks

- *recurrent neural networks* (*RNNs*) are neural networks with hidden states
- before introducing the RNN model, we first revisit the MLP model introduced in Chapter 2
- let us take a look at an MLP with a single hidden layer
- let the hidden layer's activation function be ϕ
- given a mini-batch of examples $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ with batch size n and d inputs, the hidden layer's output $\boldsymbol{H} \in \mathbb{R}^{n \times h}$ is calculated as:

$$\boldsymbol{H} = \phi(\boldsymbol{X}\boldsymbol{W}_{xh} + \boldsymbol{b}_h). \tag{2}$$

- in (2), we have the weight parameter $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$, the bias parameter $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$, and the number of hidden units h, for the hidden layer
- thus, broadcasting is applied during the summation

4.2 Recurrent neural networks

- next, the hidden variable $\boldsymbol{H}$ is used as the input of the output layer
- the output layer is given by:

$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}_{hq} + \boldsymbol{b}_q,$$

  where $\boldsymbol{O} \in \mathbb{R}^{n \times q}$ is the output variable, $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$ is the weight parameter, and $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$ is the bias parameter of the output layer
- if it is a classification problem, we can use softmax($\boldsymbol{O}$) to compute the probability distribution of the output categories
- this is entirely analogous to the regression problem we discussed previously in Section 4.1
- things are entirely different when we have hidden states
- let us look at the structure in some more detail

4.2 Recurrent neural networks

- assume that we have a mini-batch of inputs $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ at time step t
- in other words, for a mini-batch of n sequence examples, each row of $\boldsymbol{X}_t$ corresponds to one example at time step t from the sequence
- next, denote by $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$ the hidden variable of time step t
- unlike the MLP, here we save the hidden variable $\boldsymbol{H}_{t-1}$ from the previous time step and introduce a new weight parameter $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$ to describe how to use the hidden variable of the previous time step in the current time step
- specifically, the calculation of the hidden variable of the current time step is determined by the input of the current time step, together with the hidden variable of the previous time step:

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h). \tag{3}$$

4.2 Recurrent neural networks

- compared with (2), (3) adds one more term $\boldsymbol{H}_{t-1}\boldsymbol{W}_{hh}$, and thus instantiates (1)
- from the relationship between hidden variables $\boldsymbol{H}_t$ and $\boldsymbol{H}_{t-1}$ of adjacent time steps, we know that these variables captured and retained the sequence's historical information up to their current time step, just like the state or memory of the neural network's current time step
- therefore, such a hidden variable is called a *hidden state*
- since the hidden state uses the same definition of the previous time step in the current time step, the computation of (3) is *recurrent*
- hence, neural networks with hidden states based on recurrent computation are named *recurrent neural networks* (*RNNs*)
- layers that perform the computation of (3) in *RNNs* are called *recurrent layers*

4.2 Recurrent neural networks

- there are many different ways for constructing RNNs; RNNs with a hidden state defined by (3) are very common
- for time step t, the output of the output layer is similar to the computation in the MLP:

$$\boldsymbol{O}_t = \boldsymbol{H}_t \boldsymbol{W}_{hq} + \boldsymbol{b}_q.$$

- parameters of the RNN include the weights $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$, $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$, and the bias $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$ of the hidden layer, together with the weights $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$, and the bias $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$ of the output layer
- it is worth mentioning that, even at different time steps, RNNs always use these same model parameters
- therefore, the parameterization cost of an RNN does not grow as the number of time steps increases

4.2 Recurrent neural networks

- Figure 3 illustrates the computational logic of an RNN at three adjacent time steps
- at any time step t, the computation of the hidden state can be treated as:
  - concatenating the input $\boldsymbol{X}_t$ at the current time step t and the hidden state $\boldsymbol{H}_{t-1}$ at the previous time step $t-1$
  - feeding the concatenation result into a fully-connected layer with the activation function ϕ
- the output of such a fully-connected layer is the hidden state $\boldsymbol{H}_t$ of the current time step t

4.2 Recurrent neural networks

- in this case, the model parameters are the concatenation of $\boldsymbol{W}_{xh}$ and $\boldsymbol{W}_{hh}$, and a bias of $\boldsymbol{b}_h$, all from (3)
- the hidden state of the current time step t, $\boldsymbol{H}_t$, will participate in computing the hidden state $\boldsymbol{H}_{t+1}$ of the next time step $t+1$
- what is more, $\boldsymbol{H}_t$ will also be fed into the fully-connected output layer to compute the output $\boldsymbol{O}_t$ of the current time step t
- so, the calculation of $\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh}$ for the hidden state is equivalent to matrix multiplication between the concatenation of $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ and the concatenation of $\boldsymbol{W}_{xh}$ and $\boldsymbol{W}_{hh}$

4.2 Recurrent neural networks

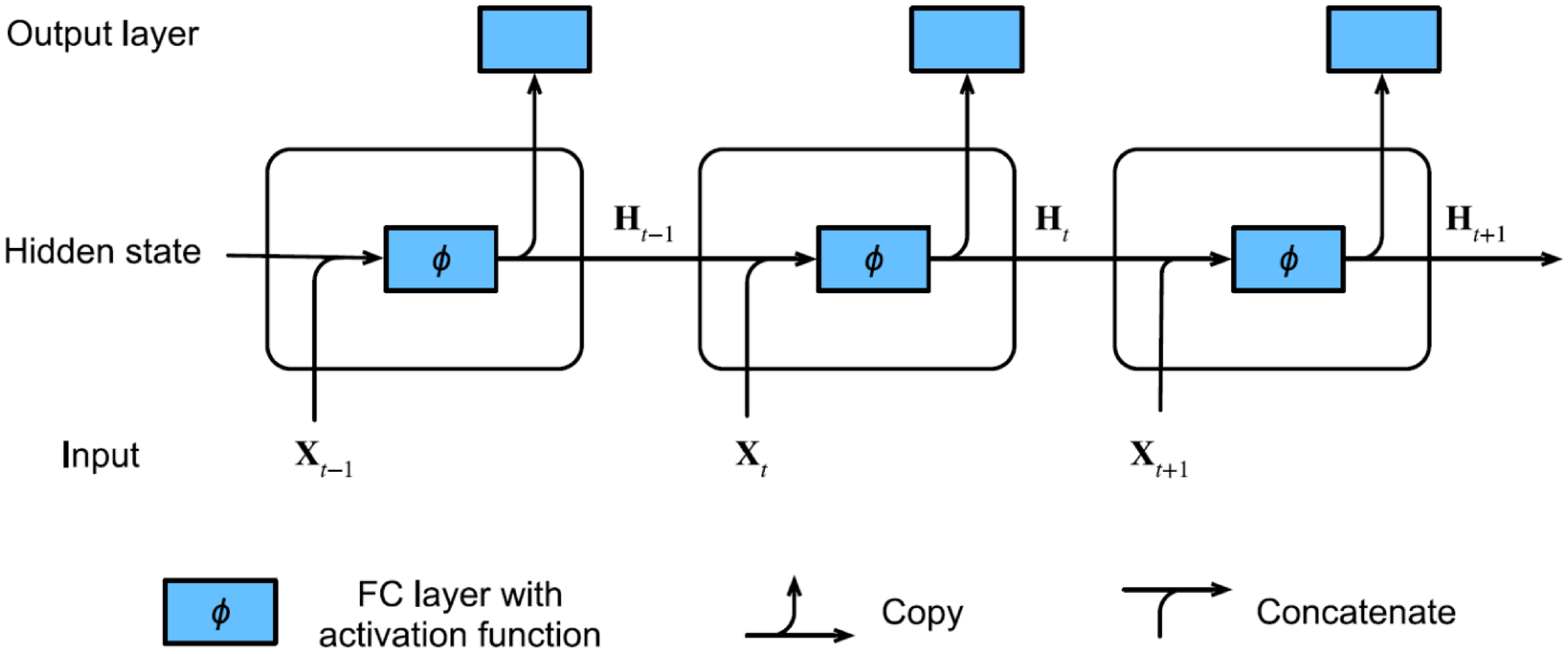


Figure 3: An RNN with a hidden state.

4.2 Recurrent neural networks

- for a sequence of length T, we will compute the gradients over T time steps in an iteration, which results in a chain of matrix products with length $\mathcal{O}(T)$, during backpropagation
- as mentioned in Chapter 2, this might result in numerical instability, e.g., the gradients may either explode or vanish, when T is large
- therefore, RNN models often need extra help to stabilize the training

4.2 Recurrent neural networks

- generally speaking, when solving an optimization problem, we take update steps for the model parameter, say in the vector form $\boldsymbol{x}$, in the direction of the negative gradient $\boldsymbol{g}$, on a mini-batch
- for example, with $\eta > 0$ as the learning rate, in one iteration, we update $\boldsymbol{x}$ as $\boldsymbol{x} - \eta \boldsymbol{g}$
- let us further assume that the objective function f is well behaved, say, *Lipschitz continuous* with constant L
- that is to say, for any $\boldsymbol{x}$ and $\boldsymbol{y}$, we have:

$$|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq L||\boldsymbol{x} - \boldsymbol{y}||.$$

4.2 Recurrent neural networks

- in this case, we can safely assume that, if we update the parameter vector by $\boldsymbol{g}$, then:

$$|f(\boldsymbol{x}) - f(\boldsymbol{x} - \eta \boldsymbol{g})| \leq L\eta||\boldsymbol{g}||,$$

  which means that we will not observe a change by more than $L\eta||\boldsymbol{g}||$
- this is both a curse and a blessing
- on the curse side, it limits the speed of making progress; whereas on the blessing side, it limits the extent to which things can go wrong if we move in the wrong direction

4.2 Recurrent neural networks

- sometimes, the gradients can be quite large, and the optimization algorithm may fail to converge
- we could address this by reducing the learning rate η
- but what if we only *rarely* get large gradients?
- in this case, such an approach may appear entirely unnecessary
- one popular alternative is to *clip* the *gradient* $\boldsymbol{g}$ by projecting it back to a ball of a given radius, say θ, via:

$$\boldsymbol{g} \leftarrow \min\left(1, \frac{\theta}{||\boldsymbol{g}||}\right)\boldsymbol{g}.$$

4.2 Recurrent neural networks

- by doing so, we know that the gradient norm never exceeds θ, and that the updated gradient is entirely aligned with the original direction of $\boldsymbol{g}$
- it also has the desirable side-effect of limiting the influence any given mini-batch (and within it, any given sample) can exert on the parameter vector
- this gives a certain degree of *robustness* to the model
- gradient clipping provides a quick fix to the gradient exploding
- while it does not entirely solve the problem, it is one of the many techniques to alleviate it

4.2 Recurrent neural networks

- recall that, for language modeling, in Section 4.1, we aim to predict the next token based on the current and past tokens, thus we shift the original sequence by one token as the labels
- Bengio et al. first proposed to use a neural network for language modeling, in 2003
- in the following, we illustrate how RNNs can be used to build a language model
- let the mini-batch size be 1, and the sequence of the text be “machine”
- to simplify things, we tokenize text into characters, rather than words, and consider a *character-level language model*
- Figure 4 demonstrates how to predict the next character based on the current and previous characters, via an RNN for character-level language modeling

4.2 Recurrent neural networks

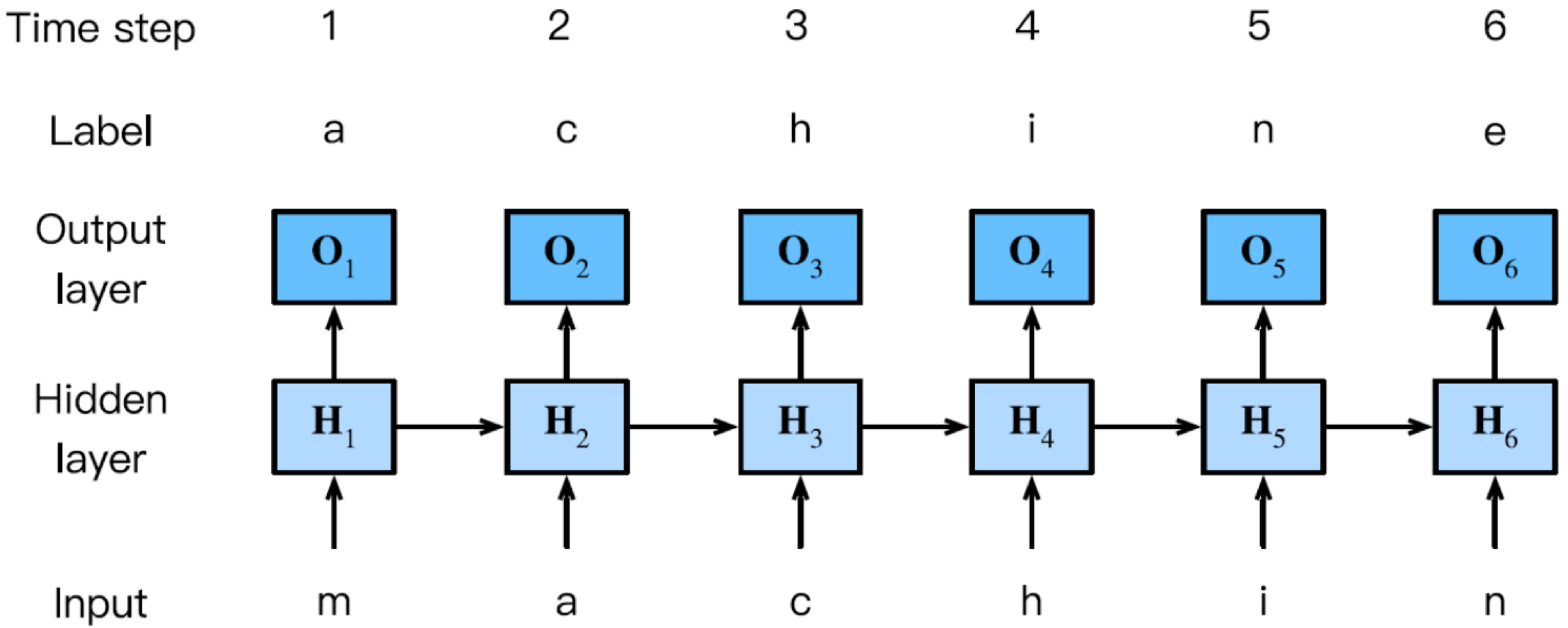


Figure 4: A character-level language model based on the RNN. The input and label sequences are “machin” and “achine”, respectively.

4.2 Recurrent neural networks

- during the training process, we run a softmax operation on the output from the output layer for each time step, and then use the cross-entropy loss to compute the error between the model output and the label
- due to the recurrent computation of the hidden state in the hidden layer, the output of time step 3 in Figure 4, $\boldsymbol{O}_3$, is determined by the text sequence “m”, “a”, and “c”
- since the next character of the sequence in the training data is “h”, the loss of time step 3 will depend on the probability distribution of the next character generated based on the feature sequence “m”, “a”, “c”, and the label “h” of this time step

4.2 Recurrent neural networks

- in practice, each token is represented by a d-dimensional vector, and we use a batch size $n > 1$
- therefore, the input $\boldsymbol{X}_t$ at time step t will be a $n \times d$ matrix, which is identical to what we discussed before
- lastly, let us discuss about how to measure the language model quality, which will be used to evaluate RNN-based models
- one way is to check how *surprising* the text is
- a good language model is able to predict with high accuracy tokens that we will see next
- consider the following continuations of the phrase “It is raining”, as proposed by different language models:
  1. “It is raining outside”
  2. “It is raining banana tree”
  3. “It is raining piouw;kcj pwepoiut”

4.2 Recurrent neural networks

- in terms of quality, example 1 is clearly the best; the words are logically coherent
- while it might not quite accurately reflect which word follows semantically (“in San Francisco” and “in winter” would have been perfectly reasonable extensions), the model is able to capture which kind of word follows
- example 2 is considerably worse, by producing a nonsensical extension
- nonetheless, at least the model has learned how to spell words, and some degree of correlation between words
- lastly, example 3 indicates a poorly trained model that does not fit the data properly

4.2 Recurrent neural networks

- we might measure the quality of the model by computing the likelihood of the sequence
- unfortunately, this is a number that is hard to understand and difficult to compare
- after all, shorter sequences are much more likely to occur than the longer ones, hence evaluating the model on a long novel will inevitably produce a much smaller likelihood than on a short novella
- what is missing is the equivalent of an average

4.2 Recurrent neural networks

- information theory is useful here; we have defined entropy and cross-entropy when we introduced the softmax regression, in Chapter 1
- if we want to compress text, we can ask about predicting the next token, given the current set of tokens
- a better language model should allow us to predict the next token more accurately
- thus, it should allow us to spend fewer bits in compressing the sequence

4.2 Recurrent neural networks

- so, we can measure it by the cross-entropy loss averaged over all the n tokens of a sequence:

$$\frac{1}{n}\sum_{t=1}^{n} -\log P(x_t|x_{t-1},\ldots,x_1), \tag{4}$$

  where P is given by a language model and x_t is the actual token observed at time step t from the sequence
- this makes the performance on documents of different lengths comparable
- for historical reasons, scientists in natural language processing prefer to use a quantity called *perplexity*
- in a nutshell, it is the exponential of (4):

$$\exp\left(-\frac{1}{n}\sum_{t=1}^{n}\log P(x_t|x_{t-1},\ldots,x_1)\right).$$

4.2 Recurrent neural networks

- perplexity can be best understood as the harmonic mean of the number of real choices that we have when deciding which token to pick next
- let us look at a number of cases:
  - In the best case scenario, the model always perfectly estimates the probability of the label token as 1. In this case, the perplexity of the model is 1.
  - In the worst case scenario, the model always predicts the probability of the label token as 0. In this situation, the perplexity is positive infinity.
  - At the baseline, the model predicts a uniform distribution over all the available tokens of the vocabulary. In this case, the perplexity equals the number of unique tokens of the vocabulary. In fact, if we were to store the sequence without any compression, this would be the best we could do to encode it. Hence, this provides a nontrivial upper bound that any useful model must beat.

4.3 Gated recurrent units (GRU)

- we have introduced the basics of RNNs, which can better handle sequence data
- however, such techniques may not be sufficient for a wide range of sequence learning problems from nowadays
- for instance, a notable issue in practice is the numerical instability of RNNs
- although we have discussed implementation tricks such as gradient clipping, this issue can be alleviated further with more sophisticated designs of sequence models
- specifically, gated RNNs are much more common in practice

4.3 Gated recurrent units (GRU)

- we will introduce two such widely-used networks, namely gated recurrent units (GRUs) and long short-term memory (LSTM)
- furthermore, we will expand the RNN architecture with a single hidden layer, which has been discussed so far, and describe deep architectures, with multiple hidden layers
- such expansions are frequently adopted in modern recurrent networks

4.3 Gated recurrent units (GRU)

- in Chapter 2, we discussed that long products of matrices can lead to vanishing or exploding gradients
- let us briefly think about what such gradient anomalies mean in practice:
  - We might encounter a situation where an early observation is highly significant for predicting all future observations. Consider the case where the first observation contains a checksum and the goal is to discern whether the checksum is correct at the end of the sequence. In this case, the influence of the first token is vital. We would like to have some mechanisms for storing vital early information in a *memory cell*. Without such a mechanism, we will have to assign a very large gradient to this observation, since it affects all the subsequent observations.
  - We might encounter situations where some tokens carry no pertinent observation. For instance, when parsing a web page, there might be auxiliary HTML code that is irrelevant for the purpose of assessing the sentiment conveyed on the page. We would like to have some mechanism for *skipping* such tokens in the latent state representation.
  - We might encounter situations where there is a logical break between parts of a sequence. For instance, there might be a transition between chapters in a book, or a transition between a bear and a bull market for stocks. In this case, it would be nice to have a means of *resetting* our internal state representation.

4.3 Gated recurrent units (GRU)

- a number of methods have been proposed to address this
- one of the earliest is long short-term memory, proposed in 1997, which we will discuss in Section 4.4
- the gated recurrent unit (GRU), proposed in 2014, is a slightly lighter variant that often offers comparable performance and is significantly faster to compute
- due to its simplicity, we will start with the GRU

4.3 Gated recurrent units (GRU)

- the key distinction between vanilla RNNs and GRUs is that the latter support gating of the hidden state
- this means that we have dedicated mechanisms for when a hidden state should be *updated*, and also when it should be *reset*
- these mechanisms are learned, and they address the concerns listed above
- for instance, if the first token is of great importance, we will learn not to update the hidden state after the first observation
- likewise, we will learn to skip irrelevant temporary observations
- lastly, we will learn to reset the latent state whenever needed; we discuss this in detail below

4.3 Gated recurrent units (GRU)

- the first thing we need to introduce are the *reset gate* and the *update gate*
- we engineer them to be vectors with entries in $(0, 1)$, such that we can perform convex combinations
- for instance, a reset gate would allow us to control how much of the previous state we might still want to remember
- likewise, an update gate would allow us to control how much of the new state is just a copy of the old state

4.3 Gated recurrent units (GRU)

- we begin by engineering these gates
- Figure 5 illustrates the inputs for both the reset and update gates in a GRU, given the input of the current time step and the hidden state of the previous time step
- the outputs of the two gates are given by two fully-connected layers with a sigmoid activation function

4.3 Gated recurrent units (GRU)

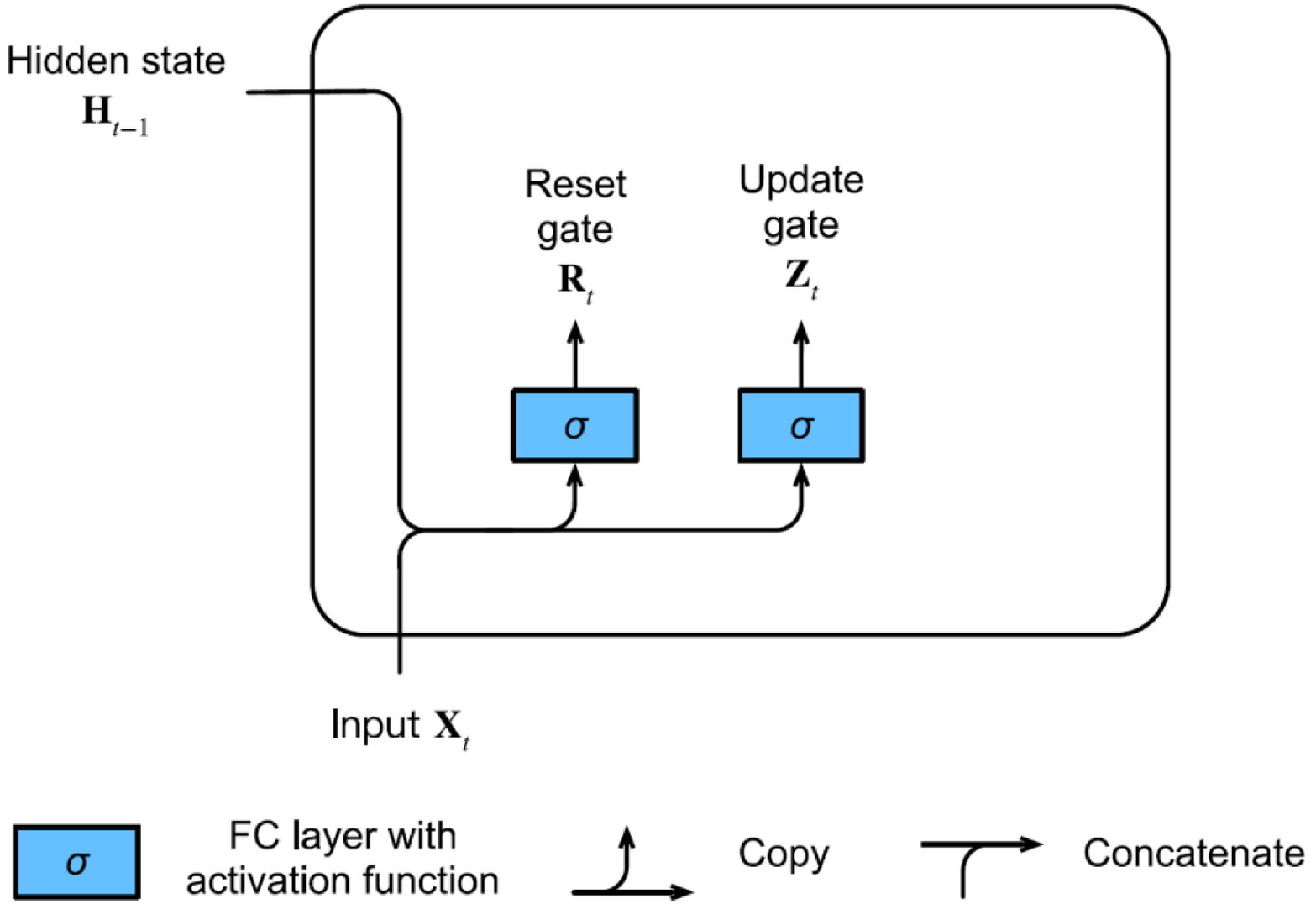


Figure 5: Computing the reset gate and the update gate in a GRU model.

4.3 Gated recurrent units (GRU)

- mathematically, for a given time step t, suppose that the input is a mini-batch $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ (number of examples: n, number of inputs: d) and the hidden state of the previous time step is $\boldsymbol{H}_{t-1} \in \mathbb{R}^{n \times h}$ (number of hidden units: h)
- then, the *reset gate* $\boldsymbol{R}_t \in \mathbb{R}^{n \times h}$ and *update gate* $\boldsymbol{Z}_t \in \mathbb{R}^{n \times h}$ are computed as follows:

$$\begin{aligned} \boldsymbol{R}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xr} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hr} + \boldsymbol{b}_r), \\ \boldsymbol{Z}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xz} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hz} + \boldsymbol{b}_z), \end{aligned}$$

  where $\boldsymbol{W}_{xr}, \boldsymbol{W}_{xz} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hr}, \boldsymbol{W}_{hz} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\boldsymbol{b}_r, \boldsymbol{b}_z \in \mathbb{R}^{1 \times h}$ are biases
- note that broadcasting is triggered during the summation
- we use the sigmoid function σ to transform input values to the interval $(0, 1)$

4.3 Gated recurrent units (GRU)

- next, let us integrate the reset gate $\boldsymbol{R}_t$ with the regular latent state updating mechanism in (3)
- it leads to the following *candidate hidden state* $\tilde{\boldsymbol{H}}_t \in \mathbb{R}^{n \times h}$ at time step t:

$$\tilde{\boldsymbol{H}}_t = \tanh(\boldsymbol{X}_t \boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1}) \boldsymbol{W}_{hh} + \boldsymbol{b}_h), \tag{5}$$

  where $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$ are weight parameters, $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$ is the bias, and the symbol $\odot$ is the Hadamard (element-wise) product operator
- here, we use a nonlinearity in the form of tanh to ensure that the values in the candidate hidden state remain in the interval $(-1, 1)$

4.3 Gated recurrent units (GRU)

- the result is a *candidate*, since we still need to incorporate the action of the update gate
- comparing with (3), now, the influence of the previous states can be reduced with the element-wise multiplication of $\boldsymbol{R}_t$ and $\boldsymbol{H}_{t-1}$ in (5)
- whenever the entries in the reset gate $\boldsymbol{R}_t$ are close to 1, we recover a vanilla RNN, such as in (3)
- for all entries of the reset gate $\boldsymbol{R}_t$ that are close to 0, the candidate hidden state is the result of an MLP with $\boldsymbol{X}_t$ as the input
- any pre-existing hidden state is thus *reset* to defaults
- Figure 6 illustrates the computational flow after applying the reset gate

4.3 Gated recurrent units (GRU)

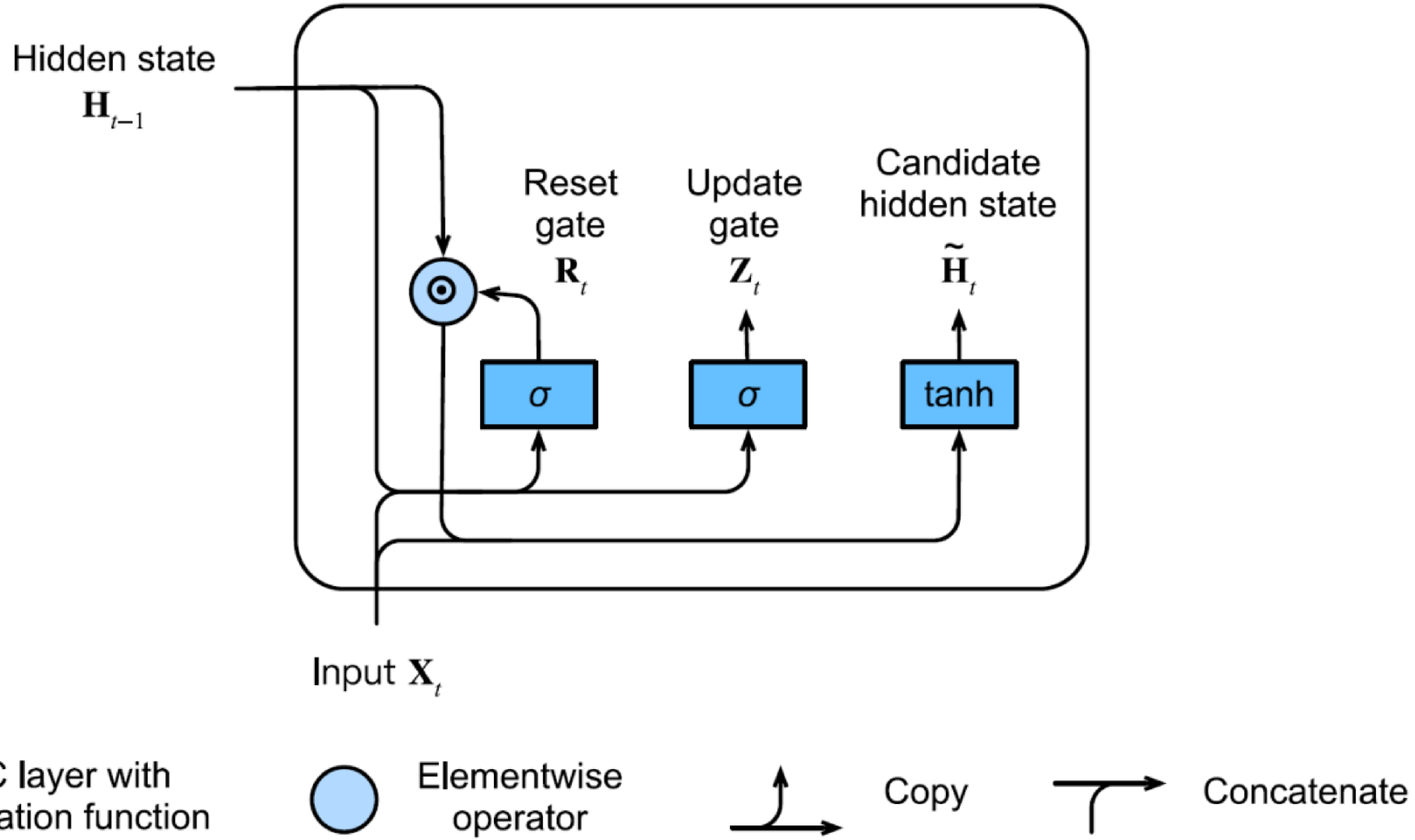


Figure 6: Computing the candidate hidden state in a GRU model.

4.3 Gated recurrent units (GRU)

- finally, we need to incorporate the effect of the update gate $\boldsymbol{Z}_t$
- this determines the extent to which the new hidden state $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$ is just the old state $\boldsymbol{H}_{t-1}$, and by how much the new candidate state $\tilde{\boldsymbol{H}}_t$ is used
- the update gate $\boldsymbol{Z}_t$ can be used for this purpose, simply by taking element-wise convex combinations between both $\boldsymbol{H}_{t-1}$ and $\tilde{\boldsymbol{H}}_t$
- this leads to the final update equation for the GRU:

$$\boldsymbol{H}_t = \boldsymbol{Z}_t \odot \boldsymbol{H}_{t-1} + (1 - \boldsymbol{Z}_t) \odot \tilde{\boldsymbol{H}}_t.$$

- whenever the update gate $\boldsymbol{Z}_t$ is close to 1, we simply retain the old state
- in this case, the information from $\boldsymbol{X}_t$ is essentially ignored, effectively skipping time step t in the dependency chain

4.3 Gated recurrent units (GRU)

- in contrast, whenever $\boldsymbol{Z}_t$ is close to 0, the new latent state $\boldsymbol{H}_t$ approaches the candidate latent state $\tilde{\boldsymbol{H}}_t$
- these designs can help us cope with the vanishing gradient problem in RNNs and better capture dependencies for sequences with large time step distances
- for instance, if the update gate has been close to 1 for all the time steps of an entire subsequence, the old hidden state at the time step of its beginning will be easily retained and passed to its end, regardless of the length of the subsequence
- Figure 7 illustrates the computational flow after the update gate is in action
- in summary, GRUs have the following two distinguishing features:
  - Reset gates help capture short-term dependencies in sequences.
  - Update gates help capture long-term dependencies in sequences.

4.3 Gated recurrent units (GRU)

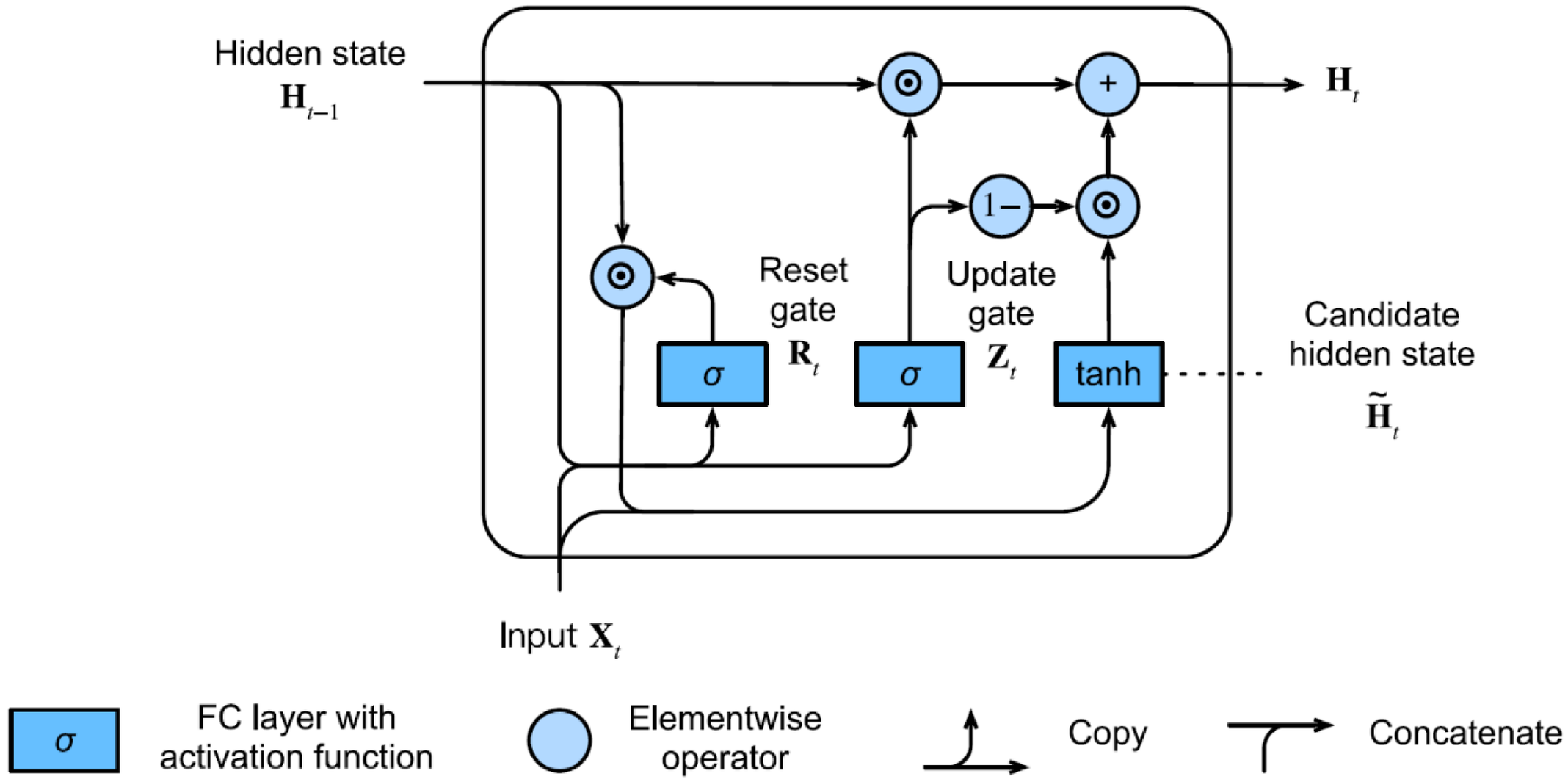


Figure 7: Computing the hidden state in a GRU model.

4.4 Long short-term memory (LSTM)

- the challenge to address long-term information preservation and short-term input skipping in latent variable models has existed for a long time
- one of the earliest approaches to address this was the *long short-term memory* (*LSTM*)
- it shares many of the properties of the GRU
- interestingly, LSTMs have a slightly more complex design than GRUs, but predate GRUs by almost two decades

4.4 Long short-term memory (LSTM)

- arguably LSTM's design is inspired by logic gates of a computer
- LSTM introduces a *memory cell* (or *cell* for short) that has the same shape as the hidden state (some papers consider the memory cell as a special type of hidden state), engineered to record additional information
- to control the memory cell, we need a number of gates
- one gate is needed to read out the entries from the cell; we will refer to this as the *output gate*

4.4 Long short-term memory (LSTM)

- a second gate is needed to decide when to read data into the cell; we refer to this as the *input gate*
- lastly, we need a mechanism to reset the content of the cell, governed by a *forget gate*
- the motivation for such a design is the same as that of GRUs, namely to be able to decide when to remember and when to ignore inputs in the hidden state, via a dedicated mechanism
- let us see how this works in practice

4.4 Long short-term memory (LSTM)

- just like in GRUs, the data feeding into the LSTM gates are the input at the current time step and the hidden state of the previous time step, as illustrated in Figure 8
- they are processed by three fully-connected layers with a sigmoid activation function to compute the values of the input, forget, and output gates
- as a result, values of the three gates are in the range of $(0, 1)$

4.4 Long short-term memory (LSTM)

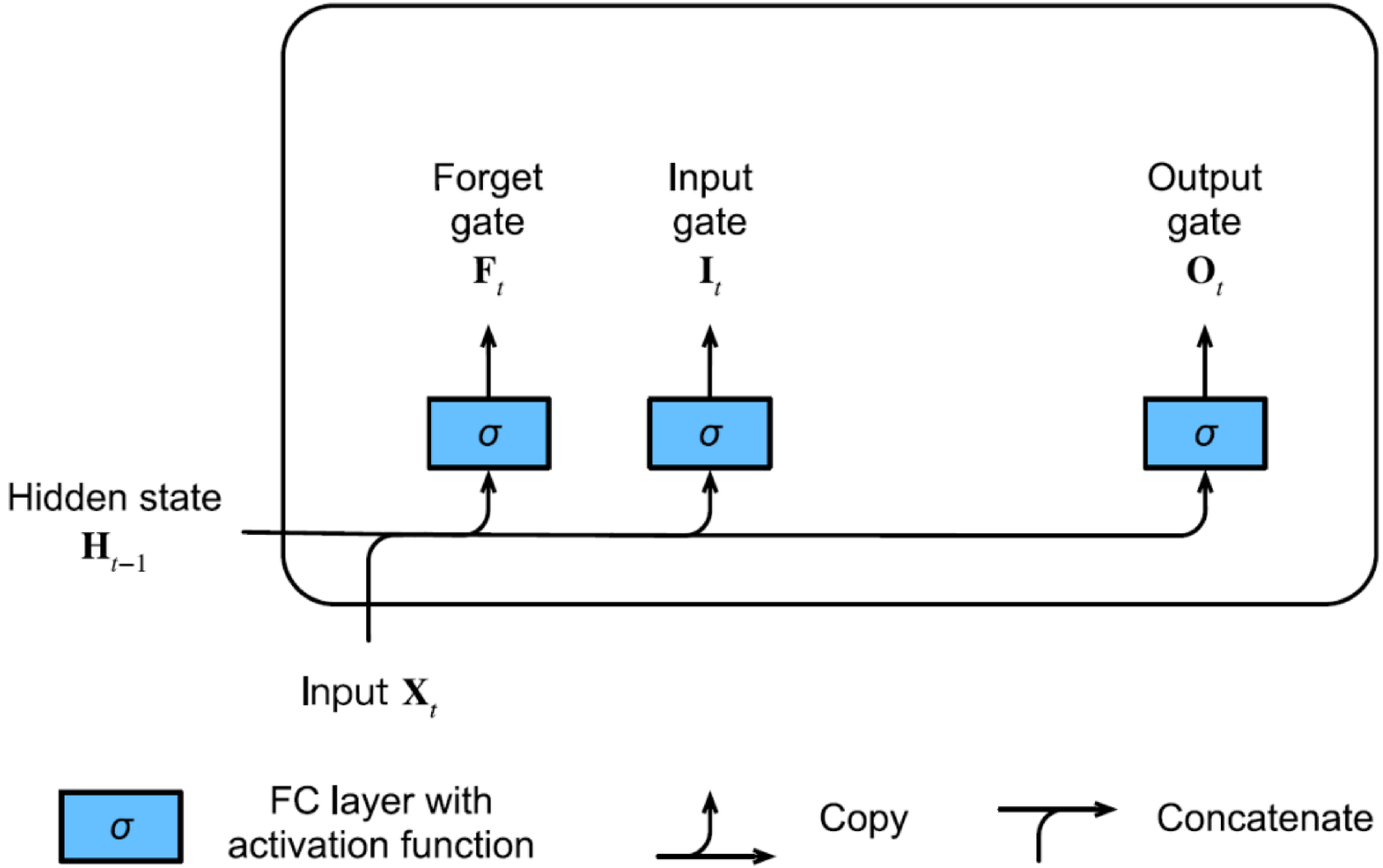


Figure 8: Computing the input gate, the forget gate, and the output gate in an LSTM model.

4.4 Long short-term memory (LSTM)

- mathematically, suppose that there are h hidden units, the batch size is n, and the number of inputs is d
- thus, the input is $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ and the hidden state of the previous time step is $\boldsymbol{H}_{t-1} \in \mathbb{R}^{n \times h}$
- correspondingly, the gates at time step t are defined as follows: the input gate is $\boldsymbol{I}_t \in \mathbb{R}^{n \times h}$, the forget gate is $\boldsymbol{F}_t \in \mathbb{R}^{n \times h}$, and the output gate is $\boldsymbol{O}_t \in \mathbb{R}^{n \times h}$
- they are calculated as follows:

$$
\begin{aligned}
\boldsymbol{I}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xi} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hi} + \boldsymbol{b}_i), \\
\boldsymbol{F}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xf} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hf} + \boldsymbol{b}_f), \\
\boldsymbol{O}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xo} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{ho} + \boldsymbol{b}_o),
\end{aligned}
$$

where $\boldsymbol{W}_{xi}, \boldsymbol{W}_{xf}, \boldsymbol{W}_{xo} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hi}, \boldsymbol{W}_{hf}, \boldsymbol{W}_{ho} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\boldsymbol{b}_i$, $\boldsymbol{b}_f, \boldsymbol{b}_o \in \mathbb{R}^{1 \times h}$ are bias parameters

4.4 Long short-term memory (LSTM)

- next, we design the memory cell
- since we have not specified the action of the various gates yet, we first introduce the *candidate* memory cell $\tilde{\boldsymbol{C}}_t \in \mathbb{R}^{n \times h}$
- its computation is similar to that of the three gates described above, but using a tanh function, with a value range of $(-1, 1)$ as the activation function
- this leads to the following equation at time step t:

$$\tilde{\boldsymbol{C}}_t = \tanh(\boldsymbol{X}_t \boldsymbol{W}_{xc} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hc} + \boldsymbol{b}_c),$$

  where $\boldsymbol{W}_{xc} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hc} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\boldsymbol{b}_c \in \mathbb{R}^{1 \times h}$ is a bias parameter
- a quick illustration of the candidate memory cell is shown in Figure 9

4.4 Long short-term memory (LSTM)

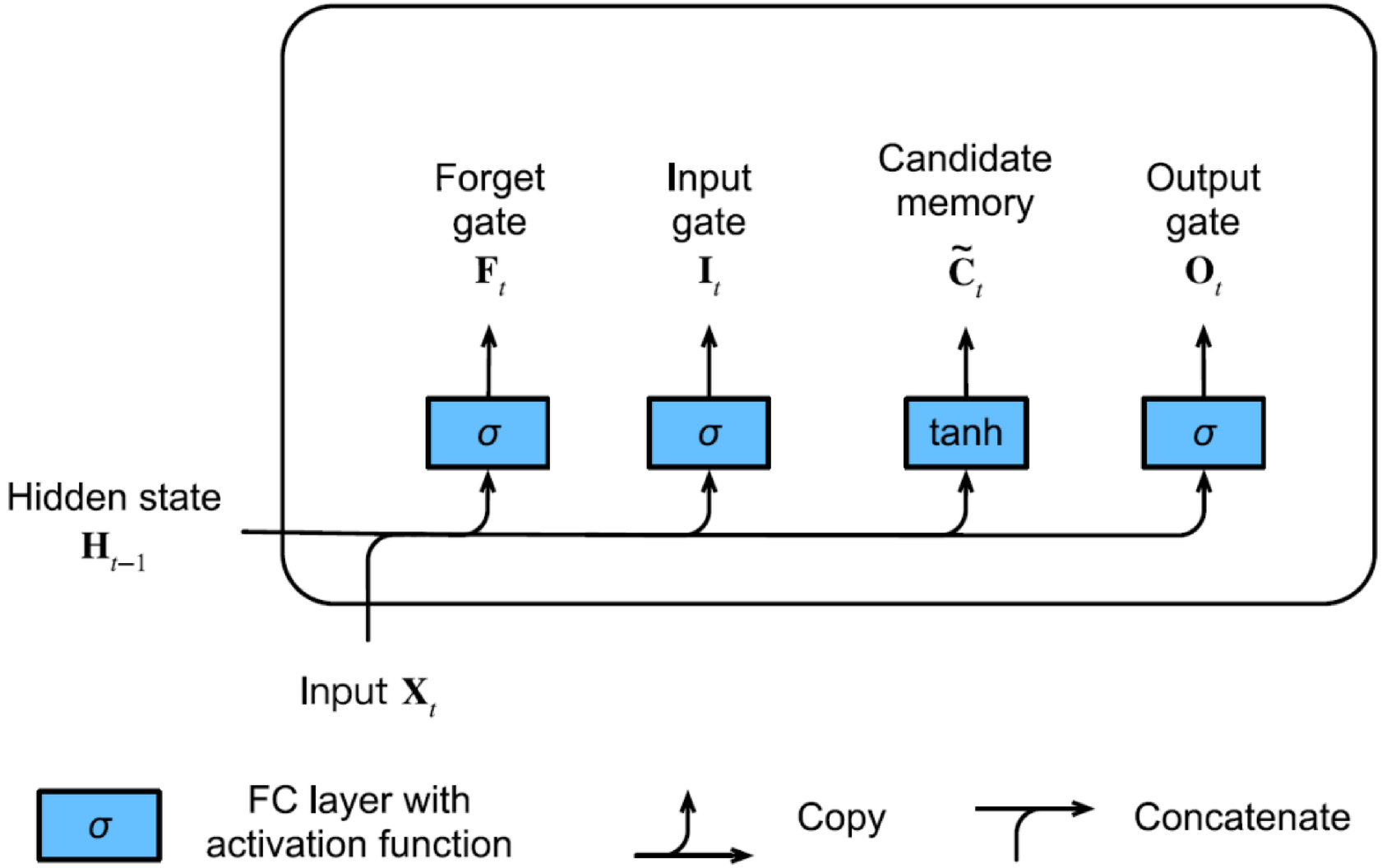


Figure 9: Computing the candidate memory cell in an LSTM model.

4.4 Long short-term memory (LSTM)

- in GRUs, we have a mechanism to govern input and forgetting (or skipping)
- similarly, in LSTMs we have two dedicated gates for such purposes: the input gate $\boldsymbol{I}_t$ governs how much we take new data into account via $\tilde{\boldsymbol{C}}_t$, and the forget gate $\boldsymbol{F}_t$ addresses how much of the old memory cell content $\boldsymbol{C}_{t-1} \in \mathbb{R}^{n \times h}$ we retain
- using the same element-wise multiplication trick as before, we arrive at the following update equation:

$$\boldsymbol{C}_t = \boldsymbol{F}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{I}_t \odot \tilde{\boldsymbol{C}}_t.$$

- if the forget gate is always approximately 1, and the input gate is always approximately 0, the past memory cells $\boldsymbol{C}_{t-1}$ will be saved over time and passed to the current time step
- this design is introduced to alleviate the vanishing gradient problem, and to better capture long range dependencies within sequences
- we thus arrive at the flow diagram in Figure 10

4.4 Long short-term memory (LSTM)

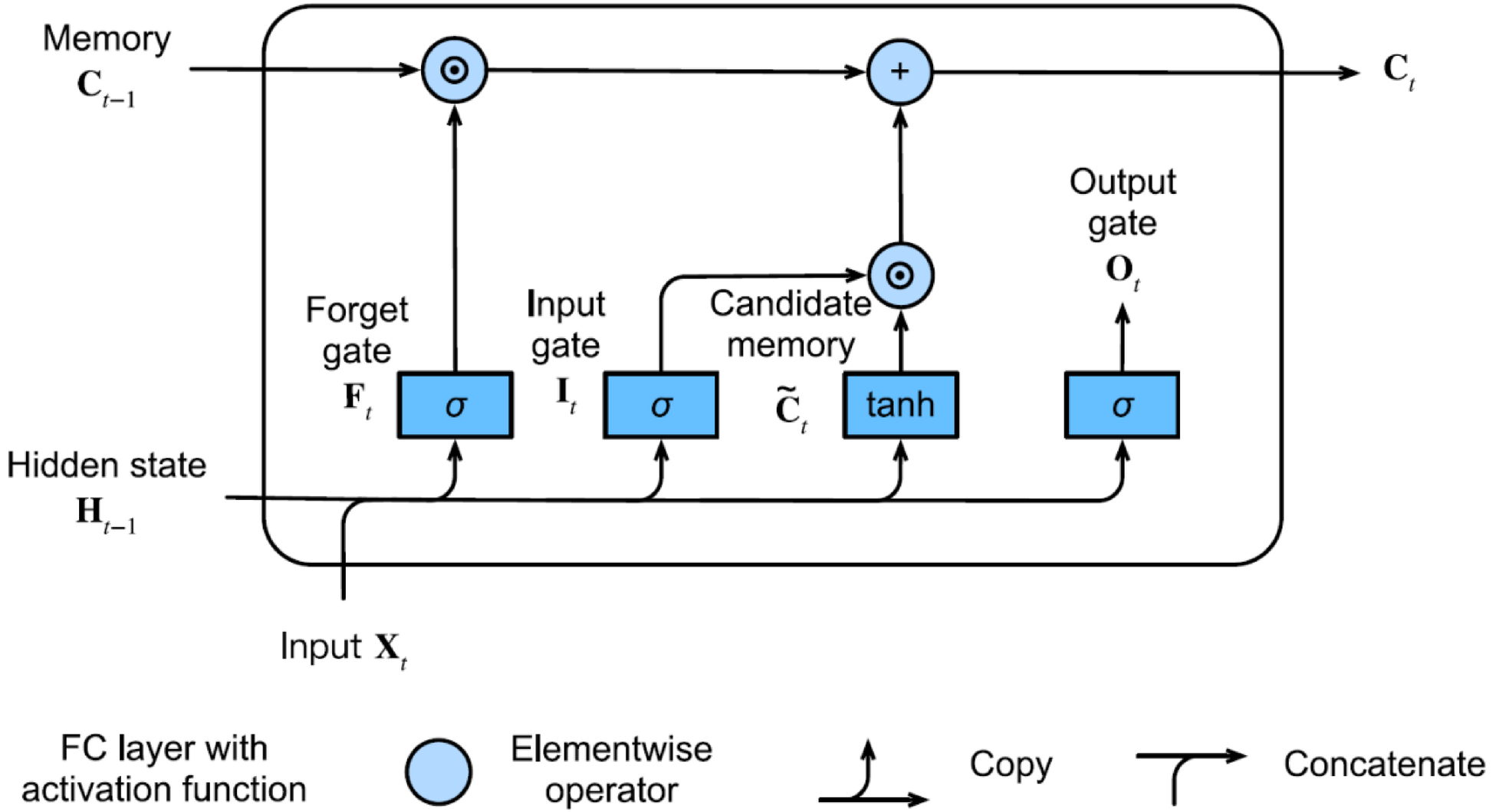


Figure 10: Computing the memory cell in an LSTM model.

4.4 Long short-term memory (LSTM)

- lastly, we need to define how to compute the hidden state $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$; this is where the output gate comes into play
- in LSTM, it is simply a gated version of the tanh of the memory cell
- this ensures that the values of $\boldsymbol{H}_t$ are always in the interval $(-1, 1)$:

$$\boldsymbol{H}_t = \boldsymbol{O}_t \odot \tanh(\boldsymbol{C}_t).$$

- whenever the output gate approximates 1, we effectively pass all memory information through to the predictor, whereas for the output gate close to 0, we retain all the information only within the memory cell, and perform no further processing
- Figure 11 gives a graphical illustration of the data flow

4.4 Long short-term memory (LSTM)

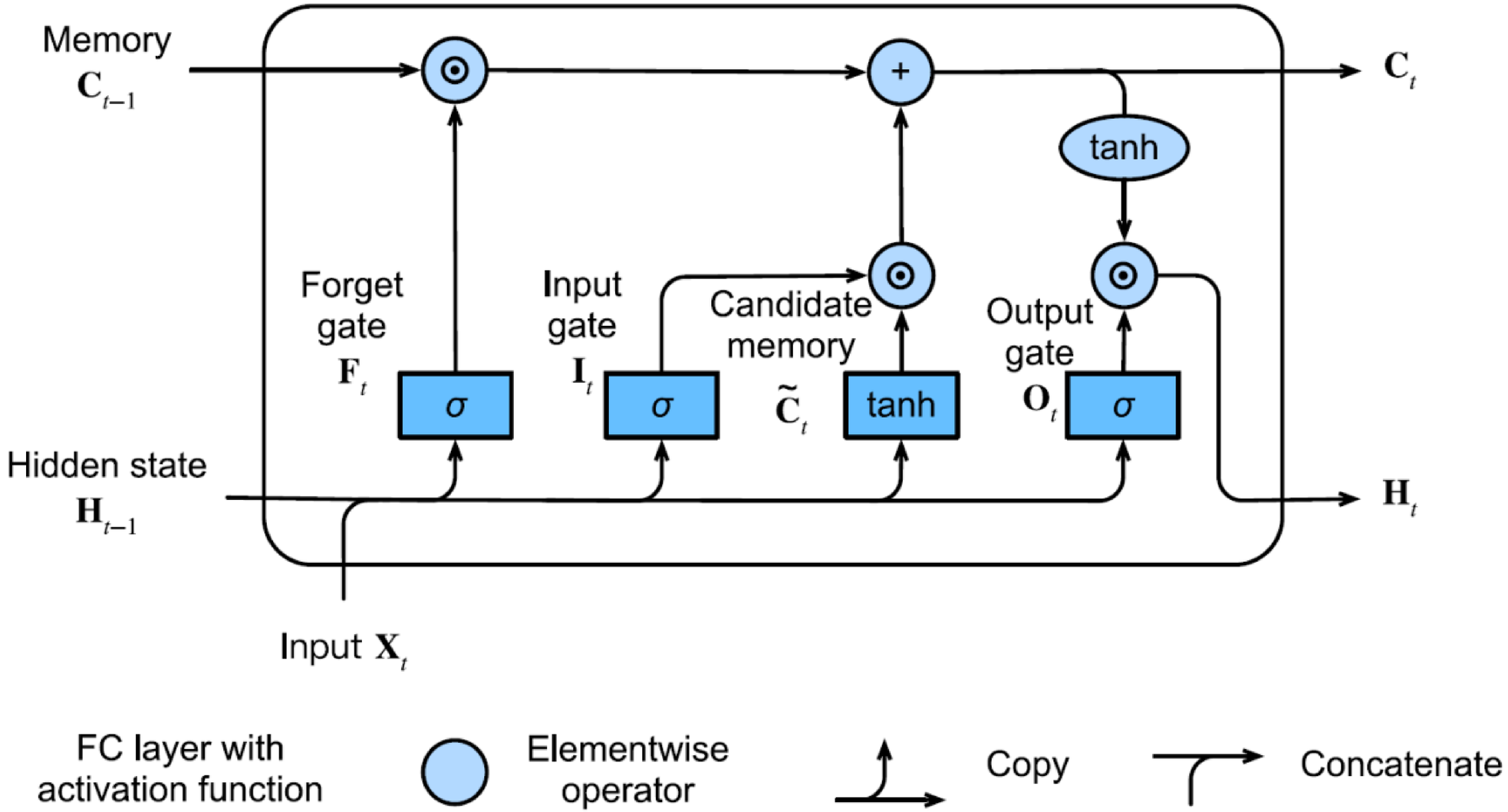


Figure 11: Computing the hidden state in an LSTM model.

4.5 Deep recurrent neural networks

- up to now, we only discussed RNNs with a single hidden layer
- in it, the specific functional form of how latent variables and observations interact is rather arbitrary
- this is not a big problem, as long as we have enough flexibility to model different types of interactions
- with a single layer, however, this can be quite challenging
- in the case of the linear models, we fixed this problem by adding more layers
- within RNNs, this is a bit trickier, since we first need to decide how and where to add extra nonlinearity

4.5 Deep recurrent neural networks

- in fact, we could stack multiple layers of RNNs on top of each other
- this results in a flexible mechanism, due to the combination of several simple layers
- in particular, data might be relevant at different levels of the stack
- for instance, we might want to keep high-level data about financial market conditions (bear or bull market) available, whereas, at a lower level, we only record shorter-term temporal dynamics
- beyond all the above abstract discussion, it is probably easiest to understand the family of models we are interested in by reviewing Figure 12; it describes a deep RNN with L hidden layers
- each hidden state is continuously passed to both the next time step of the current layer, and the current time step of the next layer

4.5 Deep recurrent neural networks

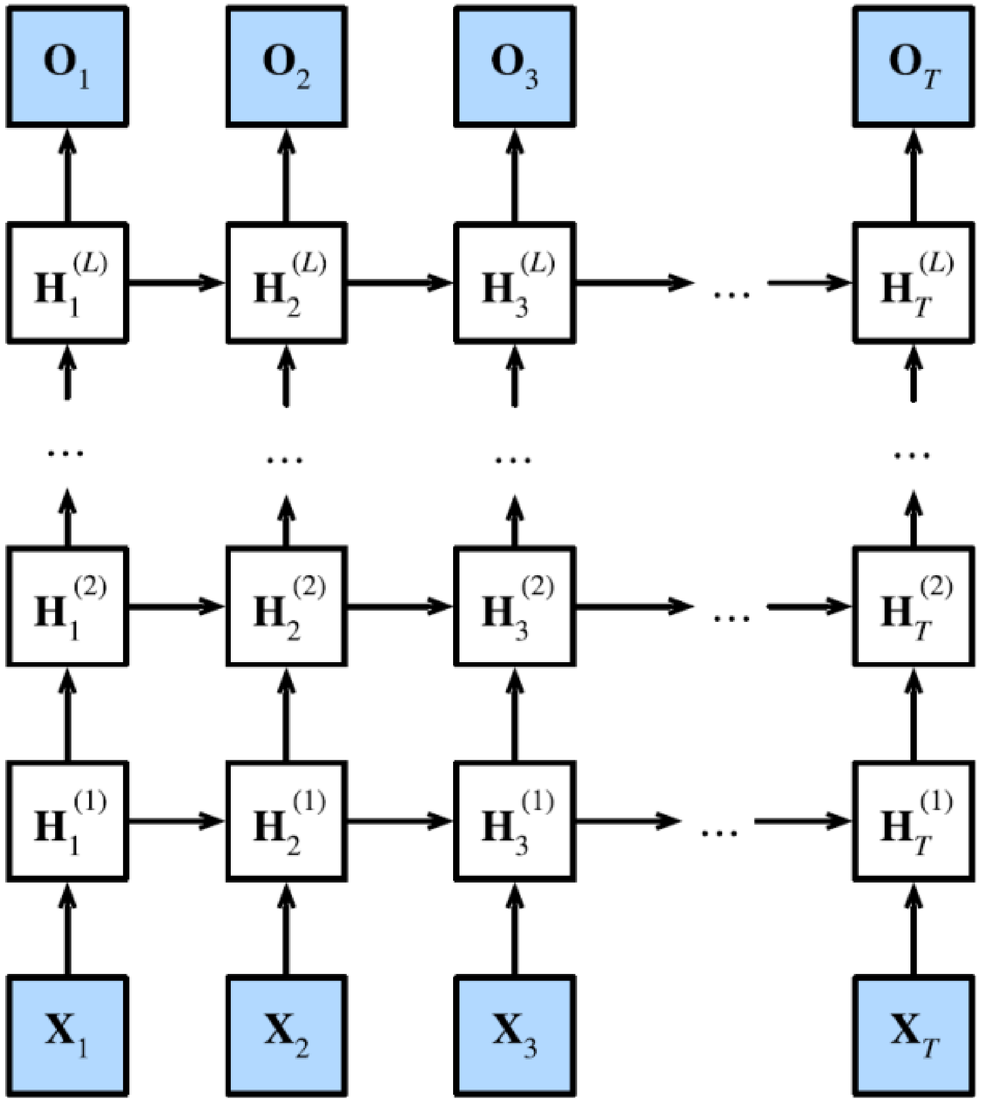


Figure 12: Architecture of a deep RNN.

4.5 Deep recurrent neural networks

- we can formalize the functional dependencies within the deep architecture of L hidden layers depicted in Figure 12
- our following discussion focuses primarily on the vanilla RNN model, but it applies to other sequence models, too
- suppose that we have a mini-batch input $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ (number of examples: n, number of inputs in each example: d) at time step t
- at the same time step, let the hidden state of the lth hidden layer ($l = 1, \ldots, L$) be $\boldsymbol{H}_t^{(l)} \in \mathbb{R}^{n \times h}$ (number of hidden units: h) and the output layer variable be $\boldsymbol{O}_t \in \mathbb{R}^{n \times q}$ (number of outputs: q)
- setting $\boldsymbol{H}_t^{(0)} = \boldsymbol{X}_t$, the hidden state of the lth hidden layer that uses the activation function ϕ_l is expressed as follows:

$$\boldsymbol{H}_t^{(l)} = \phi_l(\boldsymbol{H}_t^{(l-1)} \boldsymbol{W}_{xh}^{(l)} + \boldsymbol{H}_{t-1}^{(l)} \boldsymbol{W}_{hh}^{(l)} + \boldsymbol{b}_h^{(l)}), \tag{6}$$

where the weights $\boldsymbol{W}_{xh}^{(l)} \in \mathbb{R}^{h \times h}$ and $\boldsymbol{W}_{hh}^{(l)} \in \mathbb{R}^{h \times h}$, together with the bias $\boldsymbol{b}_h^{(l)} \in \mathbb{R}^{1 \times h}$, are the model parameters of the lth hidden layer

4.5 Deep recurrent neural networks

- in the end, the calculation of the output layer is only based on the hidden state of the final Lth hidden layer:

$$\boldsymbol{O}_t = \boldsymbol{H}_t^{(L)} \boldsymbol{W}_{hq} + \boldsymbol{b}_q,$$

  where the weight $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$ and the bias $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$ are the model parameters of the output layer
- just as with MLPs, the number of hidden layers L and the number of hidden units h are hyperparameters
- in other words, they can be tuned or specified by us
- in addition, we can easily get a deep gated RNN by replacing the hidden state computation in (6) with that from a GRU or an LSTM

4.6 Sequence to sequence learning

- RNNs are used to design language models, which are key to natural language processing
- another important benchmark is *machine translation*, a central problem domain for *sequence transduction* models that transform input sequences into output sequences
- playing a crucial role in various modern AI applications, sequence transduction models will form the focus of the remainder of this chapter and Chapter 5
- thus, we introduce the machine translation problem

4.6 Sequence to sequence learning

- *machine translation* refers to the automatic translation of a sequence from one language to another
- for decades, statistical approaches had been dominant in this field, before the rise of end-to-end learning using neural networks
- the latter is often called *neural machine translation* to distinguish itself from *statistical machine translation* that involves statistical analysis in components such as the translation model and the language model
- we will focus on neural machine translation methods
- different from language models, for which the corpus is in one single language, machine translation datasets are composed of pairs of text sequences that are in the *source language* and the *target language*, respectively

4.6 Sequence to sequence learning

- thus, machine translation is a major problem domain for sequence transduction models, whose input and output are both variable-length sequences
- to handle this type of inputs and outputs, we can design an architecture with two major components
- the first component is an *encoder*: it takes a variable-length sequence as the input and transforms it into a state with a fixed shape
- the second component is a *decoder*: it maps the encoded state of a fixed shape to a variable-length sequence
- this is called an *encoder-decoder* architecture, which is depicted in Figure 13

4.6 Sequence to sequence learning

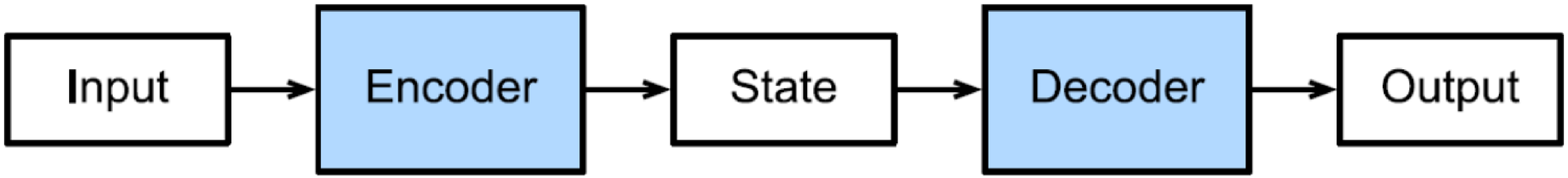


Figure 13: The encoder-decoder architecture.

4.6 Sequence to sequence learning

- let us take machine translation from English to French as an example
- given an input sequence in English: “They”, “are”, “watching”, “.”, this encoder-decoder architecture first encodes the variable-length input into a state, then decodes the state to generate the translated sequence token by token as the output: “Ils”, “regardent”, “.”
- the term “state” in the encoder-decoder architecture has probably inspired us to think about neural networks with states, when instantiating this architecture
- next, we will see how to apply RNNs to design sequence transduction models based on this encoder-decoder architecture

4.6 Sequence to sequence learning

- in machine translation, both the input and output are a variable-length sequence
- to address this type of problem, we have designed a general encoder-decoder architecture
- now, we will use two RNNs to design the encoder and the decoder of this architecture, and apply it to sequence to sequence learning for machine translation

4.6 Sequence to sequence learning

- following the design principle of the encoder-decoder architecture, the RNN encoder can take a variable-length sequence as the input, and transform it into a fixed-shape hidden state
- in other words, information of the input (source) sequence is *encoded* in the hidden state of the RNN encoder
- to generate the output sequence token by token, a separate RNN decoder can predict the next token based on what tokens have been seen (such as in language modeling) or generated, together with the encoded information of the input sequence
- Figure 14 illustrates how to use two RNNs for sequence to sequence learning in machine translation

4.6 Sequence to sequence learning

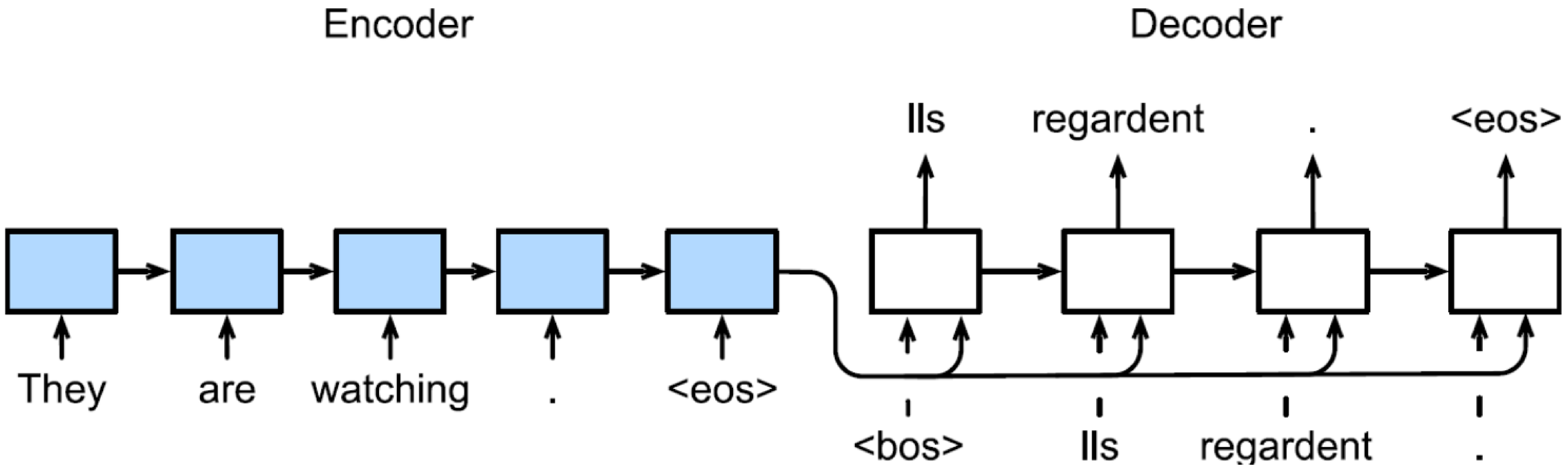


Figure 14: Sequence to sequence learning with an RNN encoder and an RNN decoder.

4.6 Sequence to sequence learning

- in Figure 14, the special “<eos>” token marks the end of the sequence
- the model can stop making predictions once this token is generated
- at the initial time step of the RNN decoder, there are two special design decisions
- first, the special beginning-of-sequence “<bos>” token is an input
- second, the final hidden state of the RNN encoder is used to initialize the hidden state of the decoder

4.6 Sequence to sequence learning

- in some designs, this is exactly how the encoded input sequence information is fed into the decoder for generating the output (target) sequence
- in some other designs, the final hidden state of the encoder is also fed into the decoder as part of the inputs at every time step, as shown in Figure 14
- similar to the training of language models, we can allow the labels to be the original output sequence, shifted by one token: “<bos>”, “Ils”, “regardent”, “.” $\rightarrow$ “Ils”, “regardent”, “.”, “<eos>”
- in the following, we will explain the design of Figure 14 in greater detail

4.6 Sequence to sequence learning

- technically speaking, the encoder transforms an input sequence of variable length into a fixed-shape *context variable* $\boldsymbol{c}$, and encodes the input sequence information in this context variable
- as depicted in Figure 14, we can use an RNN to design the encoder
- let us consider a sequence example (batch size: 1)
- suppose that the input sequence is $x_1, \ldots, x_T$, such that x_t is the tth token in the input text sequence
- at time step t, the RNN transforms the input feature vector $\boldsymbol{x}_t$ for x_t and the hidden state $\boldsymbol{h}_{t-1}$ from the previous time step into the current hidden state $\boldsymbol{h}_t$
- we can use a function f to express the transformation of the RNN's recurrent layer:

$$\boldsymbol{h}_t = f(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}).$$

4.6 Sequence to sequence learning

- in general, the encoder transforms the hidden states at all the time steps into the context variable through a customized function q:

$$\boldsymbol{c} = q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T).$$

- for example, when choosing $q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T) = \boldsymbol{h}_T$, such as in Figure 14, the context variable is just the hidden state $\boldsymbol{h}_T$ of the input sequence at the final time step
- so far, we have used an RNN to design the encoder, where a hidden state only depends on the input subsequence at, and before, the time step of the hidden state

4.6 Sequence to sequence learning

- as we just mentioned, the context variable $\boldsymbol{c}$ of the encoder's output encodes the entire input sequence $x_1, \ldots, x_T$
- given the output sequence $y_1, y_2, \ldots, y_{T'}$ from the training dataset, for each time step t' (the symbol differs from the time step t of input sequences or encoders), the probability of the decoder output $y_{t'}$ is conditional on the previous output subsequence $y_1, \ldots, y_{t'-1}$, and the context variable $\boldsymbol{c}$, i.e., $P(y_{t'}|y_1, \ldots, y_{t'-1}, \boldsymbol{c})$

4.6 Sequence to sequence learning

- to model this conditional probability on sequences, we can use another RNN as the decoder
- at any time step t' on the output sequence, the RNN takes the output $y_{t'-1}$ from the previous time step and the context variable $\boldsymbol{c}$ as its input, then transforms them and the previous hidden state $\boldsymbol{s}_{t'-1}$ into the hidden state $\boldsymbol{s}_{t'}$ at the current time step
- as a result, we can use a function g to express the transformation of the decoder's hidden layer:

$$\boldsymbol{s}_{t'} = g(y_{t'-1}, \boldsymbol{c}, \boldsymbol{s}_{t'-1}).$$

4.6 Sequence to sequence learning

- after obtaining the hidden state of the decoder, we can use an output layer and the softmax operation to compute the conditional probability distribution $P(y_{t'} | y_1, \ldots, y_{t'-1}, \boldsymbol{c})$ for the output at time step t'
- to predict the output sequence token by token, at each decoder time step, the predicted token from the previous time step is fed into the decoder as an input
- similar to training, at the initial time step, the beginning-of-sequence (“<bos>”) token is fed into the decoder
- this prediction process is illustrated in Figure 15
- when the end-of-sequence (“<eos>”) token is predicted, the prediction of the output sequence is complete

4.6 Sequence to sequence learning

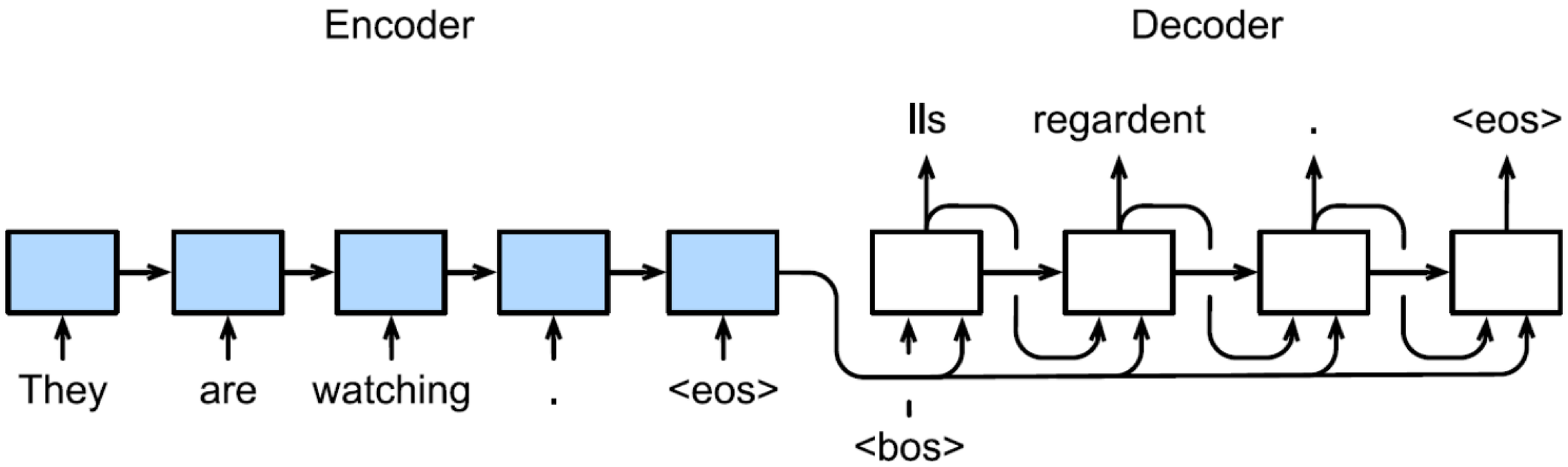


Figure 15: Predicting the output sequence token by token using an RNN encoder-decoder.

4.6 Sequence to sequence learning

- we can evaluate a predicted sequence by comparing it with the label sequence (the ground truth)
- *BLEU* (*Bilingual Evaluation Understudy*), though originally proposed for evaluating machine translation results, has been extensively used in measuring the quality of output sequences for different applications
- in principle, for any n-grams in the predicted sequence, BLEU evaluates whether these n-grams appear in the label sequence

4.6 Sequence to sequence learning

- denote by p_n the precision of n-grams, which is the ratio of the number of matched n-grams in the predicted and label sequences to the number of n-grams in the predicted sequence
- to explain, given a label sequence A, B, C, D, E, F, and a predicted sequence A, B, B, C, D, we have $p_1 = 4/5$, $p_2 = 3/4$, $p_3 = 1/3$, and $p_4 = 0$
- besides, let $\text{len}_{\text{label}}$ and len_{pred} be the number of tokens in the label sequence and the predicted sequence, respectively
- then, BLEU is defined as:

$$\exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \prod_{n=1}^{k} p_n^{1/2^n}, \tag{7}$$

where k is the longest n-grams for matching

4.6 Sequence to sequence learning

- based on the definition of BLEU in (7), whenever the predicted sequence is the same as the label sequence, BLEU is 1
- moreover, since matching longer n-grams is more difficult, BLEU assigns a greater weight to a longer n-gram precision
- specifically, when p_n is fixed, $p_n^{1/2^n}$ increases as n grows (the original paper uses $p_n^{1/n}$)
- furthermore, since predicting shorter sequences tends to obtain a higher p_n value, the coefficient before the multiplication term in (7) penalizes shorter predicted sequences
- for example, when $k = 2$, given the label sequence A, B, C, D, E, F and the predicted sequence A, B, although $p_1 = p_2 = 1$, the penalty factor $\exp(1 - 6/2) \approx 0.14$ lowers the BLEU

Exercises

Exercise 4

Define an RNN for regression, having 2 inputs, a hidden state of dimension 3, and 2 outputs. The activation function for the hidden state is ReLU. Write the equations for two time steps of forward pass through the network. For the inputs $\boldsymbol{X}_1 = \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix}$, $\boldsymbol{X}_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, if the weight matrices and bias vector are generated randomly having values from the set $\{-1, 0, 1\}$ (you can generate them yourself, not all zeros), compute the outputs $\boldsymbol{O}_1$ and $\boldsymbol{O}_2$ of the network. Specify the total number of real parameters (weights and biases) for the layer.

Exercises

Solution:

- we have the following formulas:

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h),$$

$$\boldsymbol{O}_t = \boldsymbol{H}_t \boldsymbol{W}_{hq} + \boldsymbol{b}_q,$$

where $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$, $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$, $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$, $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$, $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$, $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$, $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$, $\boldsymbol{O}_t \in \mathbb{R}^{n \times q}$, and

$$\phi(x) = \text{ReLU}(x)$$

Exercises

- in our case, $n = 2$, $d = 2$, $h = 3$, $q = 2$
- we take

$$\boldsymbol{W}_{xh} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{2\times 3},$$

$$\boldsymbol{W}_{hh} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & -1 & 0 \end{bmatrix} \in \mathbb{R}^{3\times 3},$$

$$\boldsymbol{b}_h = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \in \mathbb{R}^{1\times 3},$$

$$\boldsymbol{W}_{hq} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2},$$

$$\boldsymbol{b}_q = \begin{bmatrix} 1 & -1 \end{bmatrix} \in \mathbb{R}^{1\times 2},$$

$$\boldsymbol{H}_0 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{2\times 3}.$$

Exercises

- now, we apply the formulas:

$$
\begin{aligned}
\boldsymbol{H}_1 &= \phi\left(\begin{bmatrix}1 & 0\\ 1 & -1\end{bmatrix}\begin{bmatrix}1 & 0 & 1\\ 0 & 1 & 0\end{bmatrix} + \begin{bmatrix}0 & 0 & 0\\ 0 & 0 & 0\end{bmatrix}\begin{bmatrix}1 & 0 & -1\\ 0 & 1 & 0\\ 1 & -1 & 0\end{bmatrix} + \begin{bmatrix}1 & 0 & -1\end{bmatrix}\right)\\
&= \phi\left(\begin{bmatrix}1 & 0 & 1\\ 1 & -1 & 1\end{bmatrix} + \begin{bmatrix}0 & 0 & 0\\ 0 & 0 & 0\end{bmatrix} + \begin{bmatrix}1 & 0 & -1\\ 1 & 0 & -1\end{bmatrix}\right)\\
&= \phi\left(\begin{bmatrix}2 & 0 & 0\\ 2 & -1 & 0\end{bmatrix}\right)\\
&= \begin{bmatrix}2 & 0 & 0\\ 2 & 0 & 0\end{bmatrix} \in \mathbb{R}^{2\times 3},
\end{aligned}
$$

Exercises

$$
\begin{aligned}
\boldsymbol{H}_2 &= \phi\left(\begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}\begin{bmatrix}1 & 0 & 1\\ 0 & 1 & 0\end{bmatrix}+\begin{bmatrix}2 & 0 & 0\\ 2 & 0 & 0\end{bmatrix}\begin{bmatrix}1 & 0 & -1\\ 0 & 1 & 0\\ 1 & -1 & 0\end{bmatrix}+\begin{bmatrix}1 & 0 & -1\end{bmatrix}\right)\\
&= \phi\left(\begin{bmatrix}0 & 1 & 0\\ 1 & 0 & 1\end{bmatrix}+\begin{bmatrix}2 & 0 & -2\\ 2 & 0 & -2\end{bmatrix}+\begin{bmatrix}1 & 0 & -1\\ 1 & 0 & -1\end{bmatrix}\right)\\
&= \phi\left(\begin{bmatrix}3 & 1 & -3\\ 4 & 0 & -2\end{bmatrix}\right)\\
&= \begin{bmatrix}3 & 1 & 0\\ 4 & 0 & 0\end{bmatrix}\in\mathbb{R}^{2\times 3},
\end{aligned}
$$

Exercises

$$
\begin{aligned}
\boldsymbol{O}_1 &= \begin{bmatrix} 2 & 0 & 0 \\ 2 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \end{bmatrix} \\
&= \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \\
&= \begin{bmatrix} 3 & -1 \\ 3 & -1 \end{bmatrix} \in \mathbb{R}^{2\times 2},
\end{aligned}
$$

Exercises

$$
\begin{aligned}
\boldsymbol{O}_2 &= \begin{bmatrix} 3 & 1 & 0 \\ 4 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \end{bmatrix} \\
&= \begin{bmatrix} 3 & 1 \\ 4 & 0 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \\
&= \begin{bmatrix} 4 & 0 \\ 5 & -1 \end{bmatrix} \in \mathbb{R}^{2\times 2},
\end{aligned}
$$

$$
\begin{aligned}
\text{Total parameters} &= 2 \times 3 + 3 \times 3 + 1 \times 3 + 3 \times 2 + 1 \times 2 \\
&= 6 + 9 + 3 + 6 + 2 \\
&= 26.
\end{aligned}
$$

Vă mulţumesc!